COMMISSIONING PLATFORM

Technical Specification v5

April 2026

1. Stack

Component	Technology	Role
Device inventory	NetBox (Apache 2.0)	Source of truth. Devices, racks, cables, IPs, power chains, Config Contexts.
Device types	netbox-community/Device-Type-Library-Import	Physical specs. Imported at setup.
Telemetry	InfluxDB	Time-series store for test measurements.
WORM	MinIO with Object Lock	Immutable attested evidence.
Modbus TCP	PyModbus AsyncModbusTcpClient	Any device on port 502.
BACnet/IP	BAC0 / BACpypes3	Any device on port 47808.
SNMP	PySNMP	Any device on port 161.
IEC 61850	libiec61850 (optional)	Any device preferring 61850.
Redfish/IPMI	requests / python-ipmi	Any device with BMC.
NVML/DCGM	pynvml / DCGM REST	Any NVIDIA GPU.
REST APIs	requests	Any system with REST endpoint.
BIM/IFC	ifcopenshell	Import design spec from Revit/AutoCAD.
PDF pipeline	Anthropic API	Manufacturer PDFs → Config Contexts.
Attestation	hashlib SHA-256 + Merkle	Hash at ingestion. Chain. WORM write.
Frontend	React	Dashboard, checklist, attestation viewer.

The platform discovers devices on the network, reads their Config Context from NetBox, and communicates using whatever protocol the Config Context specifies. It does not know or care what the device does, what architecture the facility uses, or what industry it is in. Every device is the same: registers or an API endpoint, a Config Context defining how to read and test it, and an attestation chain proving the results.

2. Config Context Schema

One schema. Covers every device type. Workers are generic; Config Contexts make them device-specific. New device = new JSON file. Zero code changes.

2.1 Read Config

Field	Type	Req	Description
protocol	string	Y	modbus_tcp | bacnet_ip | snmp | redfish | rest_api | iec61850 | nvml
connection.host	string	N	If omitted, uses NetBox primary_ip.
connection.port	int	Y	502 | 47808 | 161 | 443 | 102
connection.unit_id	int	Mod	Slave ID 1-247. For devices behind gateways.
connection.timeout_seconds	int	N	Default 5. DGA/slow: 15.
connection.retries	int	N	Default 3. PyModbus exponential backoff.
connection.byte_order	string	Mod	big | little
connection.word_order	string	Mod	big | little. For 32-bit float registers.
poll_interval_seconds	int	Y	5 (fast) | 30 (normal) | 60 (slow) | 300 (DGA).
min_update_interval	int	N	Expected staleness for slow devices.
firmware_version	string	N	Expected FW. Worker warns on mismatch.
identity_register	object	N	Read for identity verification before any writes.
registers[].name	string	Y	Unique measurement name.
registers[].address	int	Mod	0-based register address.
registers[].count	int	Mod	1 (16-bit), 2 (32-bit), 4 (64-bit).
registers[].function_code	int	Mod	3 (holding) or 4 (input).
registers[].data_type	string	Y	uint16 | int16 | uint32 | int32 | float32 | float64 | boolean | bitmap
registers[].scale	float	N	Default 1.0. Raw × scale = actual.
registers[].unit	string	Y	Engineering unit.

2.2 Active Test Config

Field	Type	Description
active_tests[].name	string	Test identifier.
active_tests[].preconditions[]	array	Register reads that must pass. Safety interlocks.
active_tests[].manual_confirmation[]	string[]	Human confirmations required. Platform pauses.
active_tests[].command.address	int	Register to write.
active_tests[].command.value	int/float	Value to write.
active_tests[].command.function_code	int	6 (single) or 16 (multiple).
active_tests[].abort_conditions[]	array	If ANY triggers, immediately restore.
active_tests[].monitor[]	string[]	Register names to read during test.
active_tests[].monitor_interval_ms	int	Poll rate during test.
active_tests[].monitor_duration_seconds	int	Total monitoring window.
active_tests[].acceptance[].metric	string	max | min | avg | delta | time_to_value | settled_value | rate_of_change
active_tests[].acceptance[].operator	string	eq | neq | gt | lt | gte | lte
active_tests[].acceptance[].value	number	Threshold.
active_tests[].restore.address	int	Register to write for restore.
active_tests[].restore.value	int/float	Restore value.
active_tests[].restore_verify[]	array	Registers confirming return to normal.
active_tests[].restore_timeout_seconds	int	Max wait. Exceeding = RESTORE_FAILURE.

3. Real Config Contexts

From manufacturer documentation. Real addresses, real data types.

3.1 Schneider CM2000 Power Meter

Port 502, unit_id 1, big-endian, FC 3, poll 30s.

Reg	Name	Type	Unit
1001	frequency_hz	uint16	Hz
1002	enclosure_temp_c	uint16	°C
1003	current_phase_a	uint16	A
1004	current_phase_b	uint16	A
1005	current_phase_c	uint16	A
1008	current_3phase_avg	uint16	A
1014	voltage_ab	uint16	V
1017	voltage_ll_avg	uint16	V
1034	power_factor_total	uint16	ratio
1042	real_power_total_kw	uint16	kW
1050	apparent_power_kva	uint16	kVA

3.2 Schneider Masterpact MTZ

Via IFE Ethernet. Standard dataset at 32000+. Software-only testing.

Reg	Name	Type	Unit
32000	breaker_position	uint16	enum (0=open,1=closed,2=tripped)
32001	cradle_position	uint16	enum (0=connected,1=test,2=withdrawn)
32002	trip_cause	uint16	enum
32010	current_ia	float32 (2 reg)	A
32012	current_ib	float32 (2 reg)	A
32014	current_ic	float32 (2 reg)	A
32020	voltage_v12	float32 (2 reg)	V
32030	active_power_total	float32 (2 reg)	kW
32050	energy_active	float64 (4 reg)	kWh
32070	contact_wear_percent	uint16	%

32080	spring_charged	boolean	bit

⚠ **Float32 at 32010+ = count 2. Energy at 32050 = count 4. Wrong count = garbage.**

3.3 SEL-751 Protective Relay

Modbus TCP via Ethernet. Register map user-configurable via SET M. Defaults below.

Reg	Name	Type	Unit
0-15	relay_word_bits	bitmap	bit
100	ia_mag	float32	A
102	ib_mag	float32	A
104	ic_mag	float32	A
120	va_mag	float32	V
140	frequency	float32	Hz
150	watts_3p	float32	kW
200	breaker_ops_count	uint32	count
202	last_trip_time	float32	cycles (16.67ms at 60Hz)

⚠ **Verify against SET M config. Trip time is cycles not ms.**

3.4 Eaton InsulGard PD Monitor

Web UI + Modbus. Addresses representative; obtain from Eaton.

Reg	Name	Type	Unit
1000	pd_magnitude_max	float32	pC
1002	pd_count_per_second	uint32	count/s
1010	temperature	float32	°C
1012	humidity	float32	%RH
1020	alarm_level	uint16	enum (0-4)
1022	trend_24h	float32	pC

⚠ **Evaluate 24-72h trend, not instantaneous. Tag PD during active tests with test_id.**

3.5 Vaisala OPT100 DGA

Modbus TCP. Analysis cycle 2-10 min. Set min_update_interval: 300.

Reg	Name	Type	Unit

100	hydrogen_h2	float32	ppm
102	methane_ch4	float32	ppm
104	ethane_c2h6	float32	ppm
106	ethylene_c2h4	float32	ppm
108	acetylene_c2h2	float32	ppm
110	co	float32	ppm
112	co2	float32	ppm
120	moisture_ppm	float32	ppm
122	oil_temperature	float32	°C
124	total_gas_pressure	float32	mbar

☠️ **Acetylene > 2 ppm = active arcing. STOP all testing on that transformer.**

3.6 Qualitrol 118ITM Dry-Type Transformer Monitor

Modbus RTU via gateway. Reached as Modbus TCP through serial-to-Ethernet converter.

Reg	Name	Type	Unit
40001	winding_temp_1	float32	°C
40003	winding_temp_2	float32	°C
40005	winding_temp_3	float32	°C
40007	ambient_temp	float32	°C
40009	winding_hotspot	float32	°C (calculated)
40020	fan_stage_1	boolean	on/off
40030	alarm_status	uint16	bitmap

⚠️ **Type K thermocouples are EMI-sensitive. Use 3-reading moving average in noisy rooms.**

3.7 NVIDIA GPU (NVML/DCGM)

Metric	Source	Unit	Acceptance
Model	nvmlDeviceGetName	string	Match BOM
Serial	nvmlDeviceGetSerial	string	Match NetBox
ECC errors	nvmlDeviceGetMemoryErrorCounter	count	Zero uncorrectable
Temp (idle)	nvmlDeviceGetTemperature	°C	< 40°C
Temp (stress)	DCGM diagnostic	°C	< throttle threshold

Power (idle)	nvmlDeviceGetPowerUsage	W	< 50W
Power (stress)	DCGM sustained	W	Within 5% TDP
NVLink status	nvmlDeviceGetNvLinkState	up/down	All UP
NVLink errors	nvmlDeviceGetNvLinkErrorCounter	count	Zero
PCIe	nvmlDeviceGetCurrPcieLinkGeneration	gen/width	Per spec
Memory	nvmlDeviceGetMemoryInfo	bytes	Match spec

4. Real Active Tests

Each test is JSON in the Config Context. Test engine is generic.

4.1 UPS Battery Transfer

preconditions	battery_percent >= 80, ups_status == online, load <= 90%, input_voltage stable
manual_confirmation	None
command	Write 1 to battery_test_initiate (FC 6)
abort_conditions	output_voltage < 200V
monitor	[output_voltage, output_current, battery_voltage, battery_percent] @ 100ms, 60s
acceptance	output_voltage min >= 228V, switchover <= 10ms
restore	Write 0 to battery_test_initiate
restore_verify	ups_status == online within 30s

4.2 Breaker Trip Timing (MTZ)

preconditions	cradle == connected, spring_charged, breaker == closed
manual_confirmation	downstream_load_isolated (REQUIRED)
command	Write trip to Micrologic X (FC 6)
monitor	[breaker_position, current_ia/ib/ic, trip_cause, contact_wear] @ 50ms, 5s
acceptance	breaker == open within 83ms, trip_cause == test_command
restore	Write close command
restore_verify	breaker == closed AND spring_charged within 10s

☠️ **downstream_load_isolated is manual. Platform pauses for human confirmation.**

4.3 ATS Transfer

preconditions	gen == ready_standby, ats == source_1, ALL downstream UPS online
command	Write simulated utility failure
abort_conditions	ANY downstream ups_status != online_battery

monitor	[ats_position, gen_voltage, gen_frequency, all downstream ups_input_voltage] @ 100ms, 120s
acceptance	gen >= rated within 10s, freq 59.5-60.5Hz, ats == source_2 within 30s
restore	Write return-to-utility
restore_verify	ats == source_1, gen cooldown

☠️ **Must verify EVERY downstream UPS. One unhealthy = outage.**

4.4 Cooling Redundancy

preconditions	supply_air <= 25°C, primary running, backup standby
command	BACnet WriteProperty: shutdown primary
abort_conditions	zone_temp > 35°C
monitor	[zone_supply, zone_return, backup_fan, backup_compressor] @ 1000ms, 300s
acceptance	zone max <= 30°C, backup fan >= 60% within 30s
restore	Start primary
restore_verify	primary running, zone <= 25°C within 10 min

4.5 Coolant Loop Pressure Hold

preconditions	pressure <= 5 PSI, leak_detection normal, isolation valves closed
command	Ramp pump to rated pressure
abort_conditions	drop > 5 PSI in 60s, OR leak detected
monitor	[pressure, flow_rate, supply_temp, leak_detection] @ 500ms, 600s
acceptance	delta <= 2 PSI over 10 min, no leaks
restore	Pump to 0
restore_verify	pressure to baseline within 60s

4.6 DC Bus Battery Discharge

preconditions	soc >= 90%, bus_voltage nominal ±2%, electrical_room_doors_locked (access control API)
manual_confirmation	DC area clear of personnel (REQUIRED)

command	Open rectifier input contactor
abort_conditions	bus_voltage < minimum threshold
monitor	[bus_voltage, battery_current, soc, pdu_branches, gpu_power] @ 100ms
acceptance	bus > minimum for rated duration, zero GPU shutdowns
restore	Close rectifier contactor
restore_verify	bus nominal, battery charging

☠️ **Door lock is automated precondition via access control API. DC arc flash does not self-extinguish.**

4.7 Relay Trip (SEL-751)

preconditions	associated breaker open, relay settings match coordination study
command	Write to Remote Bit register (FC 6)
monitor	[relay_word_bits, last_trip_time, breaker_ops, SER events] @ 50ms, 5s
acceptance	50/51 pickup within 1 cycle, trip time matches study
restore	Clear Remote Bit
restore_verify	50/51 dropped out

⚠️ **Download oscillographic event report after test. Remote Bit mapping depends on SELOGIC config.**

4.8 GPU Thermal Under Load

preconditions	coolant flow at design rate, GPU idle temp < 40°C
command	DCGM: run diagnostic stress
abort_conditions	GPU Tj > throttle_threshold - 5°C
monitor	[gpu_temp, gpu_power, coolant_supply, coolant_return] @ 100ms
acceptance	Tj < throttle sustained, power within 5% TDP
restore	Stop DCGM stress
restore_verify	Tj < 45°C within 60s

⚠️ **Run AFTER coolant loop and CDU tests pass.**

5. Edge Cases

5.1 Protocol

Edge Case	Impact	Mitigation
Float32 word order mismatch	NaN/Inf/1.2e+38	byte_order + word_order. Worker detects NaN, suggests swap.
Register offset-by-one	Reads wrong register	0-based. Validate first read against expected range.
Gateway unit_id misroute	Command reaches wrong device	identity_register read BEFORE write. Mismatch = abort.
Timeout during active test	Blind during test state	Watchdog: 3 fails = blind restore + ABORTED.
Stale reads (DGA)	Same value for minutes	min_update_interval. Expected staleness.
FW-dependent register map	Config for FW 2.1 fails on 3.0	firmware_version. Warn on mismatch, skip.
IFE reboot mid-test	TCP drops	Auto-reconnect. Read state immediately on reconnect.
SEL register remap	Defaults wrong	Verify SET M config before polling.

5.2 Safety

Edge Case	Impact	Mitigation
Mechanical failure (stuck breaker)	Trip sent, didn’t trip	Monitor position. 2x expected time = MECHANICAL_FAILURE. Never auto-retry.
Restore failure (spring uncharged)	Can’t close after trip	Check spring before close. Wait charge cycle, retry once. Else RESTORE_FAILURE.
Cascading power dependency	Test A kills Test B’s device	Orchestrator models NetBox power chains. Never test shared chain simultaneously.
BMS command conflict	Operator overrides during test	Log advisory. Lock devices during test window.
Coolant spill	Fitting fails under pressure	Abort at 5 PSI drop in 60s. Never exceed rated pressure.
GPU thermal runaway	Cooling insufficient	Run AFTER cooling passes. Abort at threshold - 5°C.
PD spike during breaker test	Test transient or real defect?	Tag with test_id. Evaluate pre/post baseline only.
Network partition mid-test	Can’t push results	Local WAL. Flush on reconnect. Merkle chain continues locally.

5.3 Attestation

Edge Case	Impact	Mitigation
Clock skew	Timestamps differ	Platform NTP time. Record both. clock_skew field.
WORM write failure	Evidence lost	Local WAL. INVALID until WORM confirms. Never PASSED without WORM.
Hash chain break	Tampering or bug	Halt attestation. Critical event. Forensics required.
Duplicate records	Double attestation	Nonce dedup. Keep first, discard second.

6. Data Flow

Device → Worker → SHA-256 hash (before any DB) → Merkle chain → Dual write: InfluxDB + MinIO WORM.

6.1 Attestation Record

Field	Type
timestamp	int64 ns (platform NTP)
device_id	string (NetBox ID)
measurement	string (register name)
value	float64 (decoded, scaled)
raw_bytes	hex (for independent verification)
protocol	string
source_ip	string
worker_id	string
hash	SHA-256(timestamp+device_id+measurement+value+raw_bytes+prev_hash)
previous_hash	SHA-256
test_id	string|null

6.2 Punch List Record

Field	Description
severity	critical | major | minor | info
device	NetBox device name and ID
category	identity | network | power | cooling | security | safety | test_failure
expected	From design spec (BIM import)
actual	From discovery or test
evidence_hash	SHA-256 of attestation record
remediation	Suggested action

7. All 21 Modules

Each module independently testable. Max 500 lines. Each is one vibe-code prompt with the spec as context. Build and verify EACH before connecting to next.

BACKEND — Infrastructure (Modules 0-5)

Module 0: Docker Compose

- **What:** NetBox + PostgreSQL + Redis + InfluxDB + MinIO in one docker-compose.yml
- **Code:** 0 lines (YAML config only)
- **Test:** curl NetBox API, InfluxDB write/read, MinIO write with Object Lock, attempt delete (must fail)
- **Depends:** Nothing

Module 1: Config Context Loader

- **What:** Python script reads Config Context JSON file, POSTs to NetBox API attached to device type
- **Input:** JSON file path + device type slug
- **Output:** Config Context visible in NetBox on the device type
- **Code:** ~50 lines
- **Test:** Load CM2000 config, query API, verify registers match
- **Depends:** Module 0

Module 2: NetBox Device Reader

- **What:** Query NetBox API for active devices filtered by protocol in Config Context
- **Input:** protocol string
- **Output:** List of [{device_id, ip, config_context}, ...]
- **Code:** ~50 lines
- **Test:** Create device with config, query, verify output
- **Depends:** Module 0, 1

Module 3: Modbus TCP Poller

- **What:** Takes IP, connection config, register list. Polls via PyModbus. Returns decoded values.
- **Input:** host, port, unit_id, byte/word order, registers[]
- **Output:** Dict {register_name: decoded_scaled_value}
- **Code:** ~100 lines

- **Test:** PyModbus simulator with CM2000 registers. Verify values. Test wrong byte order → NaN detection. Test timeout → retry.
- **Depends:** PyModbus only

Module 4: InfluxDB Writer

- **What:** Takes device_id, measurements dict, timestamp. Writes as InfluxDB point.
- **Code:** ~50 lines
- **Test:** Write, query back, verify
- **Depends:** Module 0

Module 5: Attestation Engine

- **What:** Takes measurement record. SHA-256 hash chained to previous. Writes to MinIO WORM.
- **Input:** timestamp, device_id, measurement, value, raw_bytes, previous_hash
- **Output:** Attestation record in MinIO with Object Lock. Returns new hash.
- **Code:** ~150 lines
- **Test:** Write 10 chained records. Verify chain integrity. Attempt delete (must fail). Inject tamper (chain must break). Test WORM failure → WAL buffering.
- **Depends:** Module 0

BACKEND — Protocol Workers (Modules 6-9)

Module 6: Modbus Worker

- **What:** Combines Modules 2+3+4+5 into continuous poll-hash-write loop
- **Code:** ~150 lines
- **Test:** Simulator → worker → InfluxDB + MinIO end-to-end. Kill simulator mid-poll → graceful retry.
- **Depends:** Modules 2, 3, 4, 5

Module 7: BACnet/IP Poller

- **What:** Same pattern as Module 3 but using BAC0 for BACnet/IP
- **Code:** ~100 lines
- **Test:** BAC0 simulator, poll, verify
- **Depends:** BAC0

Module 8: SNMP Poller

- **What:** Same pattern using PySNMP
- **Code:** ~100 lines
- **Test:** snmpsim, poll, verify
- **Depends:** PySNMP

Module 9: NVML/DCGM Poller

- **What:** Polls GPU metrics via pynvml and DCGM REST API
- **Code:** ~100 lines
- **Test:** Mock GPU metrics endpoint, verify decode
- **Depends:** pynvml

BACKEND — Commissioning Logic (Modules 10-15)

Module 10: Active Test Engine

- **What:** Reads active_tests[] from Config Context. Executes command-measure-verify-restore with interlocks.
- **Input:** device_id, test_name
- **Output:** Test result: pass/fail per criterion. Full trace in InfluxDB. Attested in MinIO. ABORTED if watchdog triggers.
- **Code:** ~250 lines
- **Test:** Writable PyModbus simulator. Test precondition blocking (battery < 80% blocks test). Test abort trigger (voltage drops below threshold → immediate restore). Test acceptance eval. Test restore + restore_verify. Test manual_confirmation pause. Test timeout watchdog → blind restore.
- **Depends:** Modules 3, 5

Module 11: Discovery Scanner

- **What:** Sweeps CIDR range. Probes SNMP/Modbus/BACnet/Redfish per IP. Classifies against NetBox device types.
- **Input:** CIDR range
- **Output:** List of discovered devices with protocol, identity, suggested device type
- **Code:** ~200 lines
- **Test:** Multiple simulators on different ports. Verify discovery and classification.
- **Depends:** Modules 3, 7, 8

Module 12: REST API Connectors

- **What:** One connector per external system: Genetec (access), Milestone (CCTV), ServiceNow (change mgmt), Maximo (CMMS)
- **Input:** API endpoint, credentials, query params
- **Output:** Normalized event records fed through Module 5 (attestation)
- **Code:** ~100 lines per connector
- **Test:** Mock API returning sample events. Verify normalization and attestation.
- **Depends:** Module 5

Module 13: Orchestrator

- **What:** Schedules tests respecting power dependencies from NetBox circuit data. Never tests devices on shared power chain simultaneously.
- **Input:** List of tests to run
- **Output:** Sequenced execution plan. Dispatches to Module 10.
- **Code:** ~200 lines

- **Test:** Model 3-device power chain in NetBox. Submit concurrent tests on chain. Verify sequencing prevents cascade. Submit tests on independent chains. Verify parallel execution.
- **Depends:** Modules 0, 10

Module 14: Reconciliation Engine

- **What:** Compares design spec in NetBox (from BIM import) against discovered and tested actual state
- **Input:** None (reads NetBox + InfluxDB + MinIO)
- **Output:** Structured punch list JSON
- **Code:** ~200 lines
- **Test:** Import design with intentional deviations (wrong firmware, missing device, wrong rack position). Run reconciliation. Verify punch list catches all deviations with correct severity.
- **Depends:** Modules 0, 6

Module 15: Report Generator

- **What:** Produces commissioning report from punch list and attestation records
- **Input:** Punch list JSON + attestation chain reference
- **Output:** PDF report + JSON export + attestation certificate
- **Code:** ~300 lines
- **Test:** Feed sample punch list. Generate PDF. Verify content, formatting, attestation references.
- **Depends:** Module 14

BACKEND — Ingestion Pipelines (Modules 16-17)

Module 16: BIM/IFC Import

- **What:** Parses IFC file from Revit/AutoCAD. Extracts devices with XYZ coordinates, rack positions, cable runs, power paths. Creates NetBox objects via API.
- **Input:** IFC file path
- **Output:** NetBox populated with intended design state: devices, locations, connections
- **Code:** ~200 lines
- **Test:** Sample IFC file with 10 devices across 3 racks. Import. Query NetBox API. Verify all devices, positions, and connections match IFC.
- **Depends:** Module 0, ifcopenshell library

Module 17: PDF-to-Config-Context Pipeline

- **What:** Takes manufacturer equipment PDF. Sends to Anthropic API with Config Context JSON schema as prompt. LLM extracts register map. Validates against schema. Imports into NetBox.
- **Input:** PDF file path + device type slug
- **Output:** Config Context attached to device type in NetBox
- **Code:** ~300 lines
- **Test:** Feed Schneider CM2000 PDF. Verify output matches the hand-built CM2000 Config Context. Test with malformed PDF → graceful error. Test schema validation failure → reject with clear message.
- **Depends:** Module 1, Anthropic API

⚠ LLM extraction is not 100% reliable. Always validate generated Config Contexts against the original PDF before deploying to a live facility. The pipeline is a starting point, not a final product.

FRONTEND — React App (Modules 18-20)

Module 18: Dashboard

- **What:** Main commissioning UI. Shows real-time status of the entire commissioning process.
- **Views:**
- **Discovery view:** X of Y devices found, classified, matched to design. Unmatched devices highlighted. Missing devices flagged.
- **Test execution view:** Per-device test status: queued | running | passed | failed | aborted. Live register readings during active test. Progress bar per system.
- **Punch list view:** Filterable by severity, category, device, system. Click to see attestation evidence for each finding.
- **Power dependency graph:** Visual map of power chains from NetBox. Shows which tests are blocked by which. Color-coded by test status.
- **Facility overview:** High-level stats: total devices, tested, passed, failed, pending physical verification.
- **Code:** ~1,500 lines React
- **Data source:** REST API from backend (FastAPI or Flask wrapping NetBox + InfluxDB + MinIO queries)
- **Test:** Mock backend API. Verify all views render with sample data. Test filter interactions. Test live update via polling/websocket.
- **Depends:** Backend Modules 0-15 running

Module 19: Physical Verification Checklist

- **What:** Web-based checklist for the 10-20% requiring human verification. Technician opens on phone/tablet.
- **Workflow:** Technician selects device → sees checklist of physical items (bolt torque, labels, visual) → checks off each item → takes photo per item → platform timestamps, geolocates, attests (SHA-256 + WORM) each verification.
- **Features:** Offline capable (service worker). Syncs when connectivity returns. Photo compression before upload. QR code scan to select device from NetBox.
- **Code:** ~500 lines React
- **Test:** Complete checklist for mock device. Verify photos stored with attestation. Test offline → reconnect → sync. Test QR scan → device lookup.
- **Depends:** Module 5 (attestation), Module 0 (NetBox)

Module 20: Attestation Viewer

- **What:** Allows owner, lender, or auditor to independently verify any data point’s integrity.
- **Features:**
- **Search:** Find any attestation record by device, test, timestamp, or hash.

- **Chain verification:** Click any record, platform walks the Merkle chain backward and forward, highlights any broken links.
- **Evidence export:** Download attested dataset for any test as signed JSON bundle for independent verification.
- **Certificate view:** Display attestation certificate for the entire commissioning engagement.
- **Code:** ~500 lines React
- **Test:** Load sample attestation chain. Verify chain walk. Inject tampered record → verify UI shows chain break. Test export → verify JSON is valid and independently verifiable.
- **Depends:** Module 5 (attestation/MinIO)

8. Module Summary

#	Module	Lang	Lines
0	Docker Compose	YAML	0
1	Config Context Loader	Python	~50
2	NetBox Device Reader	Python	~50
3	Modbus TCP Poller	Python	~100
4	InfluxDB Writer	Python	~50
5	Attestation Engine	Python	~150
6	Modbus Worker	Python	~150
7	BACnet/IP Poller	Python	~100
8	SNMP Poller	Python	~100
9	NVML/DCGM Poller	Python	~100
10	Active Test Engine	Python	~250
11	Discovery Scanner	Python	~200
12	REST API Connectors (4x)	Python	~400
13	Orchestrator	Python	~200
14	Reconciliation Engine	Python	~200
15	Report Generator	Python	~300
16	BIM/IFC Import	Python	~200
17	PDF-to-Config Pipeline	Python	~300
18	Dashboard	React	~1,500
19	Physical Verification Checklist	React	~500
20	Attestation Viewer	React	~500

Python backend total: ~2,900 lines

React frontend total: ~2,500 lines

BIM + PDF pipelines: ~500 lines

Config Context JSON files: ~100-200 per device type, generated by Module 17

Grand total: ~5,900 lines of custom code + JSON configs. All infrastructure is open source. Zero licensing. Two people. One month.

9. Physical-Only Tasks

Cannot be automated via protocol. Handled by Module 19 (checklist with photo attestation). 10-20% of total scope.

Task	Equipment	Why
Hi-pot testing	Megger/Hipot tester	High DC voltage on de-energized conductors
Ground resistance	Fall-of-potential tester	Physical probes in earth
Transformer turns ratio	TTR test set	Physical connection to windings
Oil sampling (no online DGA)	Manual sample + lab	Only without Vaisala/Qualitrol
Bolt torque	Torque wrench	No sensor exists
Cable insulation (no monitor)	Megger	Only without embedded monitoring
Visual/cosmetic	Eyes	Labels, damage, cleanliness
Seismic bracing	Visual + measurement	Structural inspection